

House Price Prediction – Task 2

1. Objective

The objective of this project is to develop a machine learning model capable of predicting residential house prices based on various structural, spatial, and categorical features. The goal is to apply data preprocessing, feature engineering, and advanced regression techniques to build an accurate predictive system using the Kaggle House Prices dataset.

2. Tools and Technologies Used

- Python
- Kaggle Notebook Environment
- Pandas
- NumPy
- Matplotlib
- Seaborn
- SciPy
- Scikit-learn
- XGBoost
- LightGBM

3. Project Workflow

3.1 Data Collection and Loading

- Imported the House Prices dataset from Kaggle.
- Loaded training and test datasets using Pandas.
- Identified and separated the target variable (**SalePrice**).

3.2 Exploratory Data Analysis (EDA)

- Performed statistical and visual analysis of dataset features.
- Identified outliers in key variables such as **GrLivArea**.
- Visualized relationships between features and target variable.
- Analyzed missing values distribution across features.
- Studied the distribution of house prices and applied log transformation to reduce skewness.

3.3 Data Preprocessing

- Handled missing values using:
 - Mode imputation for categorical variables

- Median/zero imputation for numerical variables
 - Custom grouping-based imputation for improved accuracy
- Encoded categorical variables using one-hot encoding.
- Converted temporal features into cyclical representations.
- Scaled numerical features using RobustScaler.

3.4 Feature Engineering

New meaningful features were created to improve model performance:

- TotalArea (Living Area + Basement Area)
- TotalBathrooms (Full + Half Bathrooms combined)
- TotalPorch Area
- Binary indicators for presence of:
 - Garage
 - Pool
 - Basement
 - Fireplace
 - Second Floor

3.5 Feature Transformation

- Applied log transformation to reduce skewness in numerical features.
- Used Box-Cox transformation for highly skewed distributions.
- Ensured normalized feature distribution for improved model stability.

3.6 Model Development

Multiple machine learning models were trained and evaluated:

- Ridge Regression
- Lasso Regression
- Support Vector Regression (SVR)
- Gradient Boosting Regressor
- XGBoost Regressor
- LightGBM Regressor

3.7 Hyperparameter Optimization

- Used RandomizedSearchCV for tuning model parameters.
- Performed 5-fold cross-validation to ensure robustness.
- Optimized each model individually for best performance.

3.8 Ensemble Learning

- Built a stacking ensemble model combining multiple base learners.
- Used Ridge Regression as the final meta-learner.
- Improved generalization by combining predictions from multiple models.

3.9 Prediction and Submission

- Generated final predictions on test dataset.
- Applied inverse transformation to convert log-scale predictions back to original scale.
- Created final submission file in required Kaggle format.

4. Result

The implemented machine learning pipeline successfully predicted house prices with improved accuracy using ensemble learning techniques. The stacking model demonstrated better performance compared to individual regression models by reducing prediction error and improving generalization.

5. Conclusion

This project successfully demonstrates an end-to-end machine learning workflow, including data preprocessing, feature engineering, model training, hyperparameter tuning, and ensemble learning. The final model provides reliable and accurate predictions for house prices using the Kaggle dataset.

6. Deliverables

- Trained Machine Learning Model (Ensemble Pipeline)
- Final Prediction File ([my_prediction_ensemble.csv](#))
- Kaggle Notebook Implementation

7. Links

GitHub Repository Link:

<https://github.com/vasu-z/task-2-house-price-prediction>

Notebook Link:

<https://www.kaggle.com/code/champ1z1/task-2-house-price-prediction>

Model File Link:

<https://drive.google.com/file/d/1kczLd8dYSKvrkRWqlpEQPkbyV1yFuZU4/view>

Project Screenshots

```
import os
import math
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.gridspec as gridspec
import seaborn as sns
from scipy import stats
from scipy.stats import skew, boxcox_normmax
from scipy.special import boxcox1p

from sklearn.linear_model import Ridge, Lasso
from sklearn.svm import SVR
from sklearn.ensemble import GradientBoostingRegressor, StackingRegressor
from sklearn.metrics import mean_squared_error, make_scorer
from sklearn.model_selection import KFold, RandomizedSearchCV

sns.set_style('whitegrid')
warnings.simplefilter(action='ignore')

train_path, test_path = "", ""
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        if filename.lower() == 'train.csv':
            train_path = os.path.join(dirname, filename)
        elif filename.lower() == 'test.csv':
            test_path = os.path.join(dirname, filename)

print("Train path identified:", train_path)
print("Test path identified:", test_path)
```

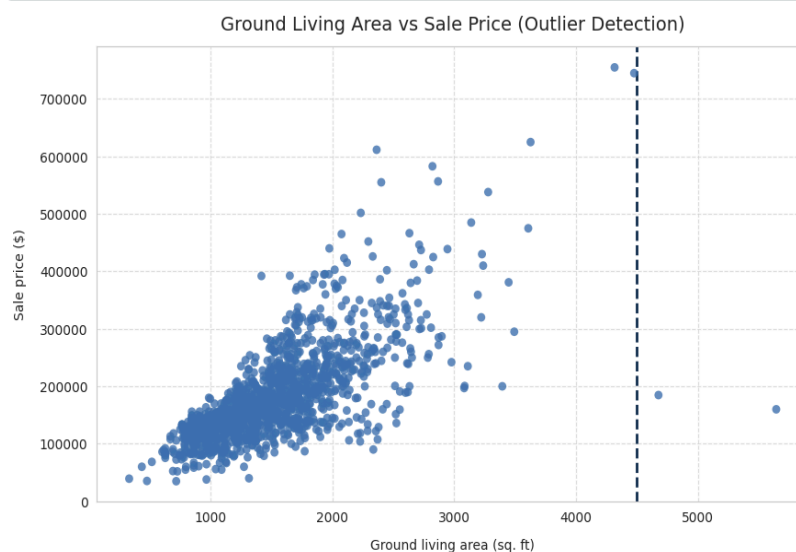
Train path identified: /kaggle/input/competitions/house-prices-advanced-regression-techniques/train.csv
Test path identified: /kaggle/input/competitions/house-prices-advanced-regression-techniques/test.csv

```
train_data = pd.read_csv(train_path)
test_data = pd.read_csv(test_path)

test_id = test_data['Id']

train_data.drop('Id', axis=1, inplace=True)
test_data.drop('Id', axis=1, inplace=True)

fig, ax = plt.subplots(figsize=(10, 6))
ax.grid(True, linestyle='--', alpha=0.6)
ax.scatter(train_data['GrLivArea'], train_data['SalePrice'], c="#3f72af", zorder=3, alpha=0.8, edgecolors='none', s=40)
ax.axvline(4500, c="#112d4e", ls="--", linewidth=2, zorder=2)
ax.set_title("Ground Living Area vs Sale Price (Outlier Detection)", fontsize=14, pad=12)
ax.set_xlabel("Ground living area (sq. ft)", labelpad=10)
ax.set_ylabel("Sale price ($)", labelpad=10)
plt.show()
```



```

train_data = train_data[train_data["GrLivArea"] < 4500]

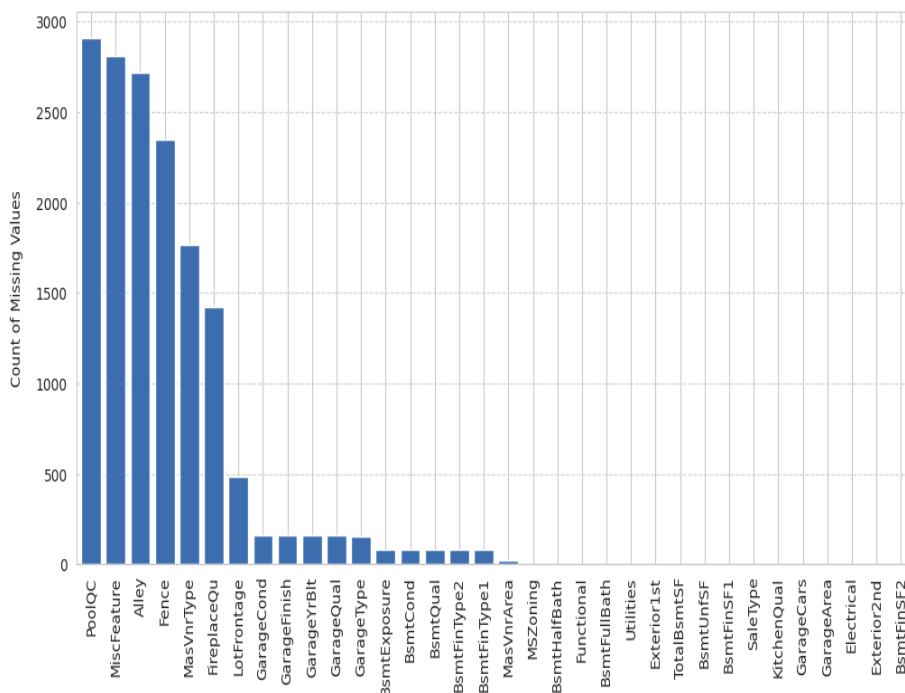
y_train_raw = train_data["SalePrice"]
data = pd.concat([train_data.drop("SalePrice", axis=1), test_data]).reset_index(drop=True)

nans = data.isna().sum().sort_values(ascending=False)
nans = nans[nans > 0]

fig, ax = plt.subplots(figsize=(12, 6))
ax.grid(axis='y', linestyle='--', alpha=0.7)
ax.bar(nans.index, nans.values, zorder=3, color="#3f72af")
ax.set_title("Distribution of Missing Data Across Features", fontsize=14, pad=12)
ax.set_ylabel("Count of Missing Values", labelpad=10)
ax.set_xlim(-0.6, len(nans) - 0.4)
ax.xaxis.set_tick_params(rotation=90, labelsiz=10)
plt.show()

```

Distribution of Missing Data Across Features



```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))

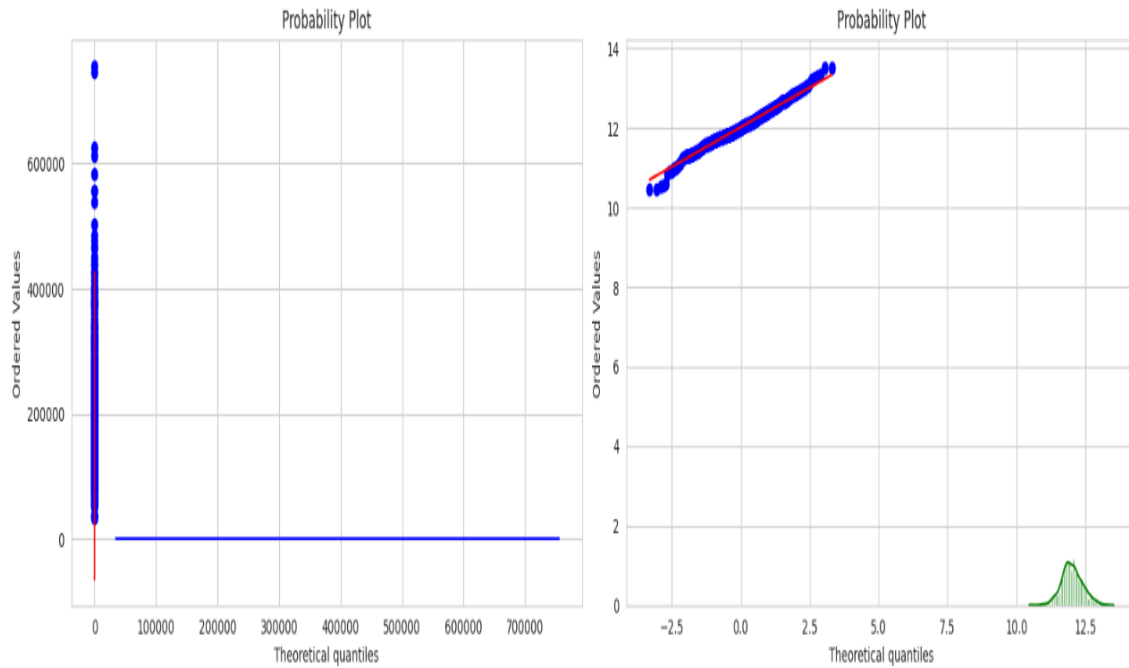
# Target variable normalization check
ax1.set_title('Original SalePrice Distribution (Skewed)', fontsize=13)
sns.histplot(y_train_raw, kde=True, ax=ax1, color='blue', stat="density")
stats.probplot(y_train_raw, plot=ax1)

# Corrected transformation assignment
y = np.log1p(y_train_raw)

# Fixed: Now passing 'y' to show the beautiful normal distribution
ax2.set_title('Log-Transformed Normal SalePrice Distribution', fontsize=13)
sns.histplot(y, kde=True, ax=ax2, color='green', stat="density")
stats.probplot(y, plot=ax2)

plt.tight_layout()
plt.show()

```



```

features_nonefill = ["PoolQC", "MiscFeature", "Alley", "Fence", "FireplaceQu", "GarageCond",
                    "GarageQual", "GarageFinish", "GarageType", "BsmtCond", "BsmtExposure",
                    "BsmtQual", "BsmtFinType2", "BsmtFinType1"]
data[features_nonefill] = data[features_nonefill].fillna("None")

features_modefill = ["MasVnrType", "MSZoning", "Utilities", "Exterior1st", "Exterior2nd", "SaleType", "Electrical", "KitchenQual", "Functional"]
for col in features_modefill:
    data[col] = data.groupby("Neighborhood")[col].transform(lambda x: x.fillna(x.mode()[0] if not x.mode().empty else "None"))

features_medianfill = ["GarageArea", "LotFrontage"]
for col in features_medianfill:
    data[col] = data.groupby("Neighborhood")[col].transform(lambda x: x.fillna(x.median()))

features_zerofill = ["GarageYrBlt", "MasVnrArea", "BsmtHalfBath", "BsmtFullBath", "BsmtFinSF1", "BsmtFinSF2", "BsmtUnfSF", "TotalBsmtSF", "GarageCars"]
data[features_zerofill] = data[features_zerofill].fillna(0)

data["TotalArea"] = data["GrLivArea"] + data["TotalBsmtSF"]
data["TotalBaths"] = data["FullBath"] + data["BsmtFullBath"] + 0.5 * (data["HalfBath"] + data["BsmtHalfBath"])
data["TotalPorch"] = data["OpenPorchSF"] + data["EnclosedPorch"] + data["3SsnPorch"] + data["ScreenPorch"]

data['Pool'] = data['PoolArea'].apply(lambda x: 1 if x > 0 else 0)
data['2ndFloor'] = data['2ndFlrSF'].apply(lambda x: 1 if x > 0 else 0)
data['Garage'] = data['GarageCars'].apply(lambda x: 1 if x > 0 else 0)
data['Bsmt'] = data['TotalBsmtSF'].apply(lambda x: 1 if x > 0 else 0)
data['Fireplace'] = data['Fireplaces'].apply(lambda x: 1 if x > 0 else 0)
data['Porch'] = data['TotalPorch'].apply(lambda x: 1 if x > 0 else 0)

data['MSSubClass', 'YrSold'] = data[['MSSubClass', 'YrSold']].astype(str)

if "MoSold" in data.columns:
    data["MoSoldsin"] = np.sin(2 * np.pi * data["MoSold"] / 12)
    data["MoSoldcos"] = np.cos(2 * np.pi * data["MoSold"] / 12)
    data = data.drop("MoSold", axis=1)
else:
    data["MoSoldsin"] = 0
    data["MoSoldcos"] = 0

print("Pipeline structures completed.")

```

Pipeline structures completed.

```

from sklearn.preprocessing import RobustScaler
import xgboost as xgb
from lightgbm import LGBMRegressor

skewed = ['LotFrontage', 'LotArea', 'MasVnrArea', 'BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF',
          'TotalBsmtSF', '1stFlrSF', '2ndFlrSF', 'GrLivArea', 'GarageArea', 'WoodDeckSF',
          'OpenPorchSF', 'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'LowQualFinSF', 'MiscVal']

skewed_features_present = [col for col in skewed if col in data.columns]
skew_features = np.abs(data[skewed_features_present].apply(lambda x: skew(x.dropna()))).sort_values(ascending=False)
high_skew = skew_features[skew_features > 0.3]

for col in high_skew.index:
    if data[col].nunique() > 1 and np.var(data[col]) > 0.001:
        try:
            data[col] = boxcox1p(data[col], boxcox_normmax(data[col] + 1))
        except Exception:
            data[col] = np.log1p(data[col])
    else:
        data[col] = np.log1p(data[col])

data = pd.get_dummies(data).copy()

numeric_cols = data.select_dtypes(include=[np.number]).columns
scaler = RobustScaler()
data[numeric_cols] = scaler.fit_transform(data[numeric_cols])

X_train = data.iloc[:len(y)].copy()
X_test = data.iloc[len(y):].copy()

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)

```

Train shape: (1458, 330)
Test shape: (1459, 330)

```

kf = KFold(n_splits=5, random_state=0, shuffle=True)
rmse = lambda y, y_pred: np.sqrt(mean_squared_error(y, y_pred))
scorer = make_scorer(rmse, greater_is_better=False)

def random_search(model, grid, n_iter=10):
    search = RandomizedSearchCV(estimator=model, param_distributions=grid, cv=kf, n_iter=n_iter, n_jobs=4, random_state=0, verbose=False)
    return search.fit(X_train, y)

xgb_hpg = {'n_estimators': [100, 400], 'max_depth': [3, 6], 'learning_rate': [0.05, 0.1]}
ridge_hpg = {'alpha': np.logspace(0, 3, 10)}
lasso_hpg = {'alpha': np.logspace(-4, -1, 10)}
svr_hpg = {'C': [1, 10], 'gamma': [0.0001, 0.001], 'epsilon': [0.01, 0.05]}
lgbm_hpg = {'colsample_bytree': [0.3, 0.5], 'learning_rate': [0.05, 0.1]}
gbm_hpg = {'max_features': [0.3, 0.5], 'learning_rate': [0.05, 0.1]}

print("Optimizing algorithms...")
xgb_search = random_search(xgb.XGBRegressor(n_jobs=4, random_state=0), xgb_hpg, n_iter=4)
ridge_search = random_search(Ridge(), ridge_hpg, n_iter=5)
lasso_search = random_search(Lasso(max_iter=15000), lasso_hpg, n_iter=5)
svr_search = random_search(SVR(), svr_hpg, n_iter=4)
lgbm_search = random_search(LGBMRegressor(random_state=0, verbose=-1), lgbm_hpg, n_iter=4)
gbm_search = random_search(GradientBoostingRegressor(random_state=0), gbm_hpg, n_iter=4)

base_models = [
    ('xgb', xgb_search.best_estimator_),
    ('ridge', ridge_search.best_estimator_),
    ('lasso', lasso_search.best_estimator_),
    ('svr', svr_search.best_estimator_),
    ('lgbm', lgbm_search.best_estimator_),
    ('gbm', gbm_search.best_estimator_)
]

models = [search.best_estimator_ for search in [xgb_search, ridge_search, lasso_search, svr_search, lgbm_search, gbm_search]]

ensemble_search = random_search(StackingRegressor(estimators=base_models, final_estimator=Ridge()), cv=kf, ('final_estimator__alpha': np.logspace(-2, 1, 10)), n_iter=3)
models.append(ensemble_search.best_estimator_)

print("All models successfully fitted.")

```

Optimizing algorithms...
All models successfully fitted.

```

prediction = [i.predict(X_test) for i in models]
predictions = np.average(prediction, axis=0)

my_prediction = pd.DataFrame({'Id': test_id, 'SalePrice': np.expml(predictions)})
my_prediction.to_csv("my_prediction_ensemble.csv", index=False)

print("File generated successfully: my_prediction_ensemble.csv")
my_prediction.head()

```

File generated successfully: my_prediction_ensemble.csv

	Id	SalePrice
0	1461	121165.156664
1	1462	157799.685691
2	1463	184107.211070
3	1464	197376.614943
4	1465	190447.830831